

SENIOR SWE INTERVIEW SERIES

Module 6: Load Balancing

Interview-focused networking, as asked at Google, Meta, Amazon, Microsoft, Uber, Netflix, Cloudflare, Stripe, LinkedIn & Cisco

Part of: Computer Networking for Senior Software Engineer Interviews

Built July 02, 2026

MODULE 6 — Load Balancing

Load balancers appear in every system design diagram. Interviewers immediately test whether yours is a cargo-cult box or an engineered decision: L4 or L7? Where does TLS end? What happens when a backend dies mid-request?

Topic 6.1 — L4 vs L7 Load Balancer

1. Why Interviewers Ask This

The single most common infra follow-up in design rounds: “what kind of LB is that box?” It tests whether you understand what information is available at each layer and what that costs.

2. Core Concept

- **L4 LB** (AWS NLB, IPVS, Maglev, Katran): balances **connections** using the 4-tuple; never parses payload; picks a backend at SYN time (or per-flow hash) and shovels packets/bytes. TLS passes through.
- **L7 LB** (ALB, NGINX, Envoy, HAProxy): **terminates** the client connection, parses HTTP, balances **requests**; can route by path/host/header, retry, cache, transform, observe. Two separate TCP connections: client ↔ LB, LB ↔ backend.

3. Internal Working

L4 modes (interviewers love these): - **NAT/full-proxy**: rewrite dst (and maybe src) IP; return traffic must come back through LB. - **DSR (Direct Server Return)**: LB forwards inbound only; backend answers the client directly with the VIP as source — LB handles ~10% of bytes (requests are small, responses big). Katran/Maglev-style. - **Consistent hashing (Maglev hashing)**: connection → backend mapping survives LB fleet changes without shared state. L7: full HTTP parse; connection pooling to backends (many client conns multiplexed onto few warm backend conns — huge win); per-request decisions; buffers/queues (watch memory).

4. Packet Flow Explanation

```
L4 (DSR): client SYN -> VIP -> [LB: hash 4-tuple -> backend B3, encap]
          -> B3 (sees client IP!) -- response --> client directly
          LB never sees response bytes. Backend picked once per conn.
L7:      client TCP+TLS -> [LB terminates] ... parses "GET /api/users"
          route rule: /api/* -> pool A (pick least-loaded, reuse warm conn)
          -> backend; response flows back THROUGH LB (can retry/observe)
```

5. ASCII Diagram

```

L4: connection-level          L7: request-level
[c1]--\                       [c1]--\   GET /a --> pool A
[c2]---> VIP hash -> B1..Bn    [c2]---> [TLS end; parse; route]
[c3]--/   no payload knowledge [c3]--/   GET /img --> pool B (+retry,
+ line-rate, cheap, protocol-      cache, auth, rate limit)
agnostic, preserves src IP         + smart; - CPU, 2x conns, new hop
(DSR); - can't route by URL,       client IP via X-Forwarded-For /
can't retry, sticky = per-conn     PROXY protocol

```

6. Real Production Example

The canonical two-tier stack — Google: Maglev (L4, consistent hashing, ECMP anycast VIPs) in front of GFE (L7, TLS termination, routing). Meta: Katran (XDP/eBPF L4) → Proxygen (L7). AWS: NLB (L4) vs ALB (L7). Cloudflare: anycast + L4 steering into L7 proxies. Interview gold: “L4 for scale and stability, L7 for brains — layered.”

7. Advantages

- L4: millions of conn/s per box, μ s latency, any protocol (databases! non-HTTP), client IP preserved (DSR/passthrough), tiny attack surface.
- L7: path/host routing, retries+timeouts+circuit breaking, TLS offload, per-request LB (beats per-connection when requests are heterogeneous — crucial for gRPC), observability (status codes, latency), WAF/authn.

8. Trade-offs

- L4: sticky per connection — one gRPC/h2 connection = one backend forever (the gRPC-through-NLB imbalance problem); no request visibility; can't retry.
- L7: CPU cost (TLS+parse), added latency (~0.5–5ms), must propagate client identity (XFF), becomes a stateful bottleneck/SPOF to engineer around, buffering risks (slowloris, big uploads).

9. Common Mistakes

- Putting gRPC behind L4 and expecting request-level balance (“one hot backend” mystery).
- Forgetting the LB → backend hop has its own keep-alive/pool/timeouts to tune.
- Claiming L7 LBs “see everything” for TCP passthrough setups, or that L4 can route by URL.
- Ignoring how the backend learns the client IP (XFF header at L7, PROXY protocol or DSR at L4).

10. Performance Impact

Rule-of-thumb: L4 $\approx$ 10–100 $\times$ the throughput per core of L7. L7 termination adds p50 ~1ms but *reduces* tail latency via retries/outlier ejection — a nuanced trade worth stating. Backend conn pooling at L7 can cut backend socket count 10–100 $\times$.

11. Common Interview Questions

1. L4 vs L7 — what can each see and do? When must you use L4? (Non-HTTP protocols, extreme scale, TLS passthrough requirements.)
2. How does a backend behind an L7 LB learn the real client IP?

3. Why is gRPC load balancing hard, and three fixes? (L7 LB, client-side LB w/ service discovery, connection rebalancing/MAX_CONNECTION_AGE.)

12. Follow-up Questions

- “How does the L4 tier itself scale beyond one box?” → ECMP/anycast from routers across an LB fleet + consistent hashing so all boxes agree.
- “What’s DSR and when can’t you use it?” → response bypasses LB; needs L2 adjacency/tunneling and no L7 features; can’t rewrite responses.
- “PROXY protocol?” → tiny header prepended on LB → backend connection carrying original 4-tuple; backend must expect it.

13. Debugging Scenarios

- One backend at 90% CPU, others idle, gRPC clients → per-connection stickiness; check connection counts per backend.
- 502s at L7 LB but backends look healthy → LB → backend idle-timeout race or pool exhaustion; align timeouts (backend > LB), check `upstream_reset` metrics.
- Client IP shows as LB IP in logs → missing XFF/PROXY handling.

14. Best Practices

- Layer them: L4 edge (scale/DDoS) → L7 (routing/retries) → services.
- Idle timeouts strictly increasing toward the backend; retries only on idempotent + budget-capped.
- For h2/gRPC: L7 balancing or client-side LB; set MAX_CONNECTION_AGE to force periodic rebalancing.

15. Practice Questions

1. Design LB for: (a) PostgreSQL, (b) public REST API, (c) internal gRPC mesh. (a: L4 only — protocol opaque + long conns; (b) L7 with WAF/TLS; (c) client-side LB or mesh sidecars.)
2. NLB → 3 backends; you deploy a 4th; no traffic reaches it for hours. Why? (Existing long-lived connections pinned; new flows only from new connections — force conn age limits.)

Topic 6.2 — Reverse Proxy

1. Why Interviewers Ask This

“LB vs reverse proxy vs API gateway” is a definitions-under-pressure question; and reverse proxies (NGINX/Envoy) are where timeouts, buffering, and header handling bugs actually live.

2. Core Concept

A reverse proxy is a server-side intermediary that accepts client requests **on behalf of** backends: one hostname in front, many services behind. Load balancing is *one feature* of a reverse proxy; others: TLS termination, caching, compression, rewriting, buffering, access control. (Forward proxy = client-side intermediary, egress; reverse = server-side, ingress. State this crisply.)

3. Internal Working

Two connections, two lifecycles: it reads the request (often buffering it fully), applies rules, opens/reuses upstream connection, streams/buffers the response. Every proxy has *three* timeout families to tune: client-side (read/idle), upstream connect, upstream read — mismatches cause 502/504s. Buffering choices: request buffering protects slow-client → backend (slowloris), but breaks streaming; response buffering frees backends fast but costs proxy memory.

4. Packet Flow Explanation

```
client ---TCP+TLS---> [proxy]          (conn 1: client's pace)
                        buffer request
                        pick upstream (pool: warm keep-alive conn)
                        add X-Forwarded-For/-Proto, Host handling
[proxy] ---http/1.1 or h2---> backend  (conn 2: LAN pace)
backend -> proxy (fast LAN) -> proxy buffers -> drips to slow client
=> backend freed in ms even for a 30s mobile download. This is the
    underrated superpower: connection/pace impedance matching.
```

5. ASCII Diagram

```

      INGRESS (reverse proxy)          EGRESS (forward proxy)
internet -> [nginx/envoy] -> svcA,svcB,svcC   corp apps -> [squid] -> internet
      TLS end, cache, LB, WAF                policy, logging
Key headers downstream->up: X-Forwarded-For (client IP chain)
                             X-Forwarded-Proto (original scheme)
                             Host / :authority (routing key)
```

6. Real Production Example

NGINX fronting app servers is the default web topology of the industry. Envoy is the modern cloud-native equivalent (Istio sidecars, Ambassador/Contour ingress) — chosen for dynamic config (xDS API), native h2/gRPC, and observability. Kubernetes Ingress controllers are reverse proxies configured via CRDs.

7. Advantages

- Single choke point for TLS, authn, rate limiting, logging (consistency).
- Slow-client isolation via buffering; backend connection pooling; zero-downtime deploys (drain upstreams); static/cache offload.

8. Trade-offs

- Extra hop (~0.2–2ms) and a component to scale/patch; buffering memory; misconfig blast radius is total (“one bad regex took the site down”); WebSocket/streaming needs explicit config (buffering off, long timeouts, Upgrade headers).

9. Common Mistakes

- Confusing forward and reverse proxies.
- Trusting `X-Forwarded-For` blindly — clients can inject it; you must trust only your own proxy chain (rightmost trusted hop) — a security-relevant detail that impresses.
- Default timeouts: NGINX `proxy_read_timeout 60s` kills long SSE/WebSockets; gRPC streams die at idle proxies.

- Forgetting `Host` /SNI handling when proxying to virtual-hosted upstreams.

10. Performance Impact

NGINX: ~100k+ RPS/core for small cached responses; proxying adds one RTT-free hop (LAN) — negligible latency, big win from pooling: 10k client conns → ~100 warm upstream conns. Response buffering can *reduce* backend fleet size materially when clients are slow (mobile).

11. Common Interview Questions

1. Reverse vs forward proxy? LB vs reverse proxy?
2. Why put NGINX in front of an app server that can serve HTTP itself? (TLS, buffering/slow clients, static, pooling, security, deploys.)
3. Walk the header hygiene: what must a proxy add/strip? (Add XFF/XFP; strip hop-by-hop headers — Connection, Keep-Alive, TE, Upgrade... except when upgrading; strip inbound spoofed XFF at the edge.)

12. Follow-up Questions

- “How do WebSockets traverse a proxy?” → HTTP/1.1 Upgrade must be explicitly forwarded (`Connection: upgrade`), buffering off, long read timeouts (Module 8).
- “What are hop-by-hop vs end-to-end headers?” → hop-by-hop consumed per-connection, must not be blindly forwarded (smuggling vectors).

13. Debugging Scenarios

- 502 Bad Gateway: proxy couldn't get a valid response — backend down/reset/timeout on connect; check upstream error counters.
- 504: upstream read timeout — backend slow; align `proxy_read_timeout` vs app SLA.
- Random disconnects of streaming endpoints at exactly 60s → default read timeout signature.

14. Best Practices

- Explicit timeout budget per route (streaming vs REST differ!); disable buffering for streams; health-check upstreams; cap request body size at the edge.
- Sanitize forwarded headers at the trust boundary; log both connection IDs (front/back) for traceability.

15. Practice Questions

1. Your SSE endpoint sends events every 90s. Users disconnect at 60s. Which knob, which component? (Proxy read/idle timeout; either heartbeat every <60s — better — or raise timeout.)
2. Design the header/trust model for client IP through: CDN → LB → NGINX → app, such that rate limiting by IP is unspoofable.

Topic 6.3 — API Gateway

1. Why Interviewers Ask This

Microservices design rounds require an entry tier; interviewers probe whether you can distinguish gateway responsibilities from LB/proxy ones — and whether you'll gold-plate it into a monolith-at-the-edge.

2. Core Concept

An API gateway is a **reverse proxy specialized for API management**: authentication/authorization (JWT/OAuth validation, API keys), rate limiting & quotas per consumer, request/response transformation, routing/versioning, API composition/aggregation (BFF), developer-facing concerns (docs, keys, billing). Think: reverse proxy = HTTP plumbing; gateway = *API product policy*.

3. Internal Working

Request pipeline (Kong/Apigee/AWS API GW/Envoy+filters all converge on this): **TLS → route match → authn (verify JWT sig/introspect token) → authz (scopes) → rate limit (local token bucket + shared store like Redis for global limits) → transform (headers/body/version shim) → upstream call(s) → response transform → metrics/billing events**. Rate limiting detail worth knowing: local buckets with async sync (fast, approximate) vs centralized counters (exact, +1 hop latency) — interviewers ask this trade-off directly.

4. Packet Flow Explanation

```
mobile app -> [API GW]
  1. verify JWT (cached JWKS public key; no auth-service hop on hot path)
  2. rate limit: user 123 -> bucket check (local, synced)
  3. route: GET /v1/profile -> profile-svc (v1->v2 shim applied)
  4. optionally aggregate: profile-svc + prefs-svc + avatar-svc
     (BFF pattern: 1 client round trip instead of 3 on mobile RTT)
  5. response: strip internal headers, add rate-limit headers, emit metrics
```

5. ASCII Diagram

```
clients          [ API GATEWAY ]          services
mobile ---\      authn | ratelimit | route  /-> users-svc
web -----+-----> transform | aggregate | -----+--> orders-svc
partners -/      version | billing | metrics \-> search-svc
vs LB: LB spreads load; GW enforces API policy. GW usually SITS ON a
proxy/LB engine (Envoy/NGINX) – they're layers, not rivals.
```

6. Real Production Example

Netflix's Zuul (and its BFF "API" layer) pioneered the pattern: device-specific endpoints aggregating dozens of microservices per screen. Amazon API Gateway fronts Lambda everywhere. Kong/Apigee dominate enterprise API monetization (keys, quotas, billing).

7. Advantages

- Centralizes cross-cutting API concerns (services stay clean); per-consumer policy (free vs paid tiers); client simplification (BFF aggregation saves mobile RTTs); versioning shims decouple client and service release trains.

8. Trade-offs

- Latency (+1–10ms per policy hop, more if authz calls out); a fat gateway becomes a shared deploy bottleneck + political choke point (“the new ESB” anti-pattern); aggregation couples the gateway to service schemas; one more thing to scale for the whole company.

9. Common Mistakes

- Putting business logic in the gateway (orchestration beyond simple aggregation → distributed monolith).
- Calling the auth service per request instead of validating JWTs locally with cached keys.
- One global gateway for internal + external traffic (internal service-to-service should use mesh/direct — don’t hairpin through the edge).
- Rate limiting only globally, not per-consumer/per-route.

10. Performance Impact

Well-built (Envoy-based) gateway: +1–3ms p50. JWT verify $\approx$ 20–100 μ s (RSA verify) cached-key. The BFF aggregation win dominates on mobile: 3 sequential 120ms RTTs → 1 (client-side) = –240ms — cite this arithmetic.

11. Common Interview Questions

1. API gateway vs load balancer vs reverse proxy vs service mesh — draw the boundaries.
2. Design rate limiting for a public API (per-key, tiers, distributed counting, 429 + Retry-After, token bucket vs sliding window).
3. Where do you validate JWTs and how do you avoid an auth bottleneck?

12. Follow-up Questions

- “Gateway vs service mesh?” → gateway = north-south (edge, client-facing policy); mesh = east-west (service-to-service mTLS/retries via sidecars). They complement.
- “How does the gateway stay stateless?” → externalize counters (Redis), cache JWKS, no sessions → horizontal scale behind an L4 LB.

13. Debugging Scenarios

- All services’ p99 degrades simultaneously → gateway saturation (CPU on TLS/JSON transforms) or its Redis rate-limit store slow; check gateway-internal latency breakdown.
- 401s spike after a key rotation → JWKS cache TTL vs rotation timing; serve old+new keys overlapping (kid-based).

14. Best Practices

- Keep it thin: authn, limits, routing, light transforms. Business orchestration belongs in services/BFFs owned by client teams.
- Fail-open vs fail-closed decision per policy (rate limiter store down: usually fail-open with alerting; authn: always fail-closed) — articulating this earns senior points.

15. Practice Questions

1. Free tier 10 rps, paid 1000 rps, 5 gateway replicas, exact-ish enforcement: design the limiter. (Local token buckets + Redis sync every ~100ms, or Redis cell/Lua atomic; discuss burst allowance + accuracy/latency trade.)
2. Mobile home screen calls 6 APIs. Design the BFF: what moves into it, what must not, and its failure semantics (partial responses with fallbacks vs all-or-nothing).

Topic 6.4 — Sticky Sessions

1. Why Interviewers Ask This

It's a judgment probe wearing a feature's clothes: interviewers want you to explain how stickiness works — then push you to say *why stateless + shared store usually beats it*.

2. Core Concept

Sticky sessions (session affinity) route the same client to the same backend every time — because that backend holds in-memory state (session, cache, WebSocket, local shopping cart). Mechanisms: **cookie-based** (LB injects **AWSALB**-style cookie → exact backend mapping; L7 only), **IP-hash** (L4-friendly; breaks with NAT/mobile), **consistent hashing on a key** (userID → shard; the legitimate modern use).

3. Internal Working

Cookie flow: first request → LB picks backend by normal algorithm → sets cookie encoding backend identity (encrypted/duration-limited) → subsequent requests carry cookie → LB routes accordingly, *bypassing* load-based choice. IP-hash: $\text{hash}(\text{srcIP}) \bmod N$ — instantly skewed by corporate NATs (one IP = 10k users) and remaps ~everyone when N changes (unless consistent hashing: only ~1/N keys move).

4. Packet Flow Explanation

```
req1: client -> LB -> (least-conn pick) -> B2 ; resp + Set-Cookie: srv=B2
req2..n: cookie srv=B2 -> LB routes to B2 regardless of B2's load
B2 dies: LB detects (health check) -> rehomes client to B4
        -> B4 has no session -> user logged out / cart empty
        (unless sessions replicated or external) <- THE core weakness
Autoscaling in: new B5 gets only NEW clients; existing load stays skewed.
```

5. ASCII Diagram

sticky:	stateless + shared store:
c1 ~~~~ B1 [c1's session]	c1 --\ /-> any backend
c2 ~~~~ B2 [c2's session]	c2 ---+> LB ->+> any backend
c3 ~~~~ B2 [c3's session]	c3 --/ \-> any backend
B2 dies => c2,c3 lose state	__ session in Redis/JWT __/
load skew, painful deploys	any backend dies: nobody notices

6. Real Production Example

Legacy Java estates (session-in-Tomcat) ran on sticky ALBs for years — and their operational pain (draining nodes for deploys took hours; scale-in lost carts) is exactly why the industry norm became external session stores (Redis/Memcached) or signed cookies/JWT. Remaining *legitimate* stickiness:

WebSocket servers (connection is inherently stateful), stateful shards/caches via consistent hashing (e.g., routing by userID to a cache-warm shard — done deliberately at Slack/Discord-scale systems).

7. Advantages

- Zero-cost session reads (local RAM); cache locality (per-user hot data); required for connection-oriented protocols; no shared-store dependency.

8. Trade-offs

- Backend death = state loss for its users; load skew (long-lived heavy users pool up); autoscaling ineffective for existing traffic; deploys need long drains; cookie stickiness breaks with cookie-blocking clients; IP stickiness breaks with CGNAT/mobility.

9. Common Mistakes

- Reaching for stickiness to “fix” an app that keeps state in memory — treating the symptom; interviewers want you to name the stateless refactor.
- IP-hash behind another proxy/CDN (all traffic from few CDN IPs → one backend gets everything).
- Assuming stickiness survives backend replacement (it doesn't; plan session durability separately).

10. Performance Impact

Local session read ~100ns vs Redis ~0.5–1ms — real but rarely decisive; the availability/elasticity costs usually dominate. Consistent-hash locality can cut cache-store QPS by 10× — the quantified defense of *deliberate* affinity.

11. Common Interview Questions

1. How do sticky sessions work at L4 vs L7?
2. What breaks when a sticky backend dies / you deploy / you autoscale?
3. Alternatives, and when is affinity actually right?

12. Follow-up Questions

- “Consistent hashing vs cookie stickiness — different problems?” → cookie: session continuity per client; consistent hashing: deterministic key → shard placement surviving topology change; the latter is architecture, the former is usually a crutch.
- “How do you drain a sticky backend for deploy?” → stop new assignments, wait for sessions to expire/migrate (hours!) — vs stateless: drain in seconds. This contrast is the interview punchline.

13. Debugging Scenarios

- One backend OOMs repeatedly while others idle → stickiness + heavy-user skew; check per-backend session counts.
- Users randomly logged out during scale-in events → affinity to terminated instances; move sessions to Redis or enable session replication as stopgap.

14. Best Practices

- Default: stateless services + external session store or signed-cookie sessions; use short sticky durations only as migration bridge.
- If affinity is required (WebSockets, shards): consistent hashing, explicit rebalancing story, per-node session budgets, graceful drain protocol.

15. Practice Questions

1. Migrate a sticky-session monolith to stateless without a big bang. (Dual-write sessions to Redis, read-through fallback to local, shrink sticky TTL, then disable affinity.)
2. Chat service, 1M WebSockets across 50 nodes. Node #17 must be replaced. Design the migration. (Drain: stop new conns, send reconnect-hints in-protocol with jitter, clients reconnect through LB to remaining nodes, presence state in shared store.)

Topic 6.5 — Health Checks

1. Why Interviewers Ask This

Health checking is where availability is actually won or lost: bad health checks cause *self-inflicted* outages (mass-unhealthy cascades) — a scenario interviewers deliberately set up.

2. Core Concept

Health checks decide the LB's routing set. Types: **active** (LB probes endpoint every N sec: TCP open / HTTP GET /healthz expecting 200) and **passive** (outlier detection: watch real traffic for 5xx/timeouts, eject misbehaving backends). Semantics split (Kubernetes vocabulary, now industry-standard): **liveness** ("restart me if stuck"), **readiness** ("route traffic to me?"), **startup** (grace during boot).

3. Internal Working

Active check state machine: `healthy --(fail_threshold consecutive fails)--> unhealthy --(success_threshold passes)--> healthy`. Detection time $\approx$ interval $\times$ fail_threshold (e.g. 10s $\times$ 3 = 30s worst case). Passive/outlier: eject on consecutive 5xx or latency percentile deviation, with max-ejection-percent cap (Envoy: default cap 10%) so you never eject the whole fleet — this cap is *the* detail that prevents cascade disasters.

4. Packet Flow Explanation

```
LB: every 10s -> GET /healthz on each backend (separate conns! bypasses
    normal pools - a backend can pass checks yet fail real traffic)
B3: deadlocked threadpool -> /healthz times out x3 -> ejected at ~30s
    meanwhile passive detection saw B3's real-request timeouts at ~2s
    and pre-ejected it => passive catches what active misses, faster.
Recovery: B3 restarts -> startup probe grace -> readiness passes ->
    slow-start ramp (LB sends it 10%->100% over 60s, avoiding cold-cache hammering)
```

5. ASCII Diagram

```

deep vs shallow checks:
/healthz (shallow): "process up, event loop alive"      -> for LIVENESS
/ready (medium): "deps connected, caches warm,         -> for READINESS
                  not overloaded, not draining"
DANGER (deep-check trap): /healthz checks DB ->
DB blips 5s -> ALL backends report unhealthy -> LB ejects ALL ->
total outage from a blip. Rule: liveness NEVER checks dependencies;
readiness may - with damping + eject caps.

```

6. Real Production Example

This exact cascade (shared dependency in health check → fleet-wide ejection) features in public postmortems across the industry and is why Envoy defaults “panic threshold”: if >50% of a cluster looks unhealthy, ignore health checks and route to everyone (better degraded than dead). Kubernetes’ three-probe model exists because early users put DB checks in liveness probes and got restart storms.

7. Advantages

- Automatic failure removal in seconds (vs minutes for DNS); enables zero-downtime deploys (readiness gating during rollout); slow-start protects cold instances.

8. Trade-offs

- Check traffic itself costs ($N \text{ LBs} \times M \text{ backends} \times \text{frequency}$ — at scale this is real QPS); false positives eject healthy capacity during load spikes (health timeout < overloaded response time → death spiral: eject → more load on rest → more ejections); checks test the *check path*, not necessarily user paths.

9. Common Mistakes

- Deep dependency checks in liveness probes (restart storms) or fleet-shared deps in readiness (mass ejection).
- Health endpoint doing real work (DB query per probe × every LB × 5s = accidental load).
- Detection-time math ignored: “we have health checks” but $\text{interval} \times \text{threshold} = 90\text{s}$ of full errors.
- No slow-start: recovered instance gets full traffic on cold caches/JIT → immediately unhealthy again (flapping).

10. Performance Impact

Availability math: with 30s detection and 10 backends, one backend hard-down costs ~10% errors × 30s per incident. Passive detection cuts this to ~seconds. Panic thresholds/eject caps bound the worst case. These numbers turn “add health checks” into engineering.

11. Common Interview Questions

1. Liveness vs readiness vs startup — and what belongs in each.
2. Design health checking so a DB outage doesn’t take down the stateless tier.
3. Active vs passive checks — why you want both.

12. Follow-up Questions

- “What should /ready return during graceful shutdown?” → non-200 *first* (drain), keep serving in-flight, then exit — ordering matters for zero-downtime deploys.
- “How do you avoid thundering recovery?” → slow-start/warmup weighting, jittered re-add.

13. Debugging Scenarios

- Flapping backend (healthy ↔ unhealthy every minute) → check timeout too close to p99 under load; raise timeout, add damping.
- Deploy causes 30s of 503s → readiness gate passes before app truly warm, or drain not honored; verify preStop/drain sequencing vs LB deregistration delay.

14. Best Practices

- Liveness: trivial self-check. Readiness: local resources + drain state; shared deps only with caps/panic thresholds. Separate endpoint from real traffic but *also* enable passive outlier detection.
- Tune the triangle: interval × threshold = detection time vs false-positive rate; add slow-start on re-add.

15. Practice Questions

1. Compute worst-case error volume: 20 backends, one dies, checks every 15s × 3 fails, 2000 rps. (~5% of 2000 rps × 45s ≈ 4,500 failed requests; then propose passive ejection to cut it ~10×.)
2. Write the /ready logic (pseudocode) for a service with Redis (cache, optional) and Postgres (critical): which failures flip readiness and why. (Postgres down → not ready *with hysteresis* + *cap awareness*; Redis down → still ready, degrade to slow path.)

Topic 6.6 — Failover

1. Why Interviewers Ask This

“What happens when X dies?” is the interviewer’s favorite pointer at every box in your diagram. Failover is the composite answer: detection + redirection + state + capacity, at each tier.

2. Core Concept

Failover = automatic redirection from failed to healthy components, layered by blast radius: - **Backend fails** → LB health checks reroute (seconds). - **LB node fails** → LB fleet behind ECMP/anycast, or VIP moves (VRRP/GARP) (sub-second to seconds). - **Zone fails** → multi-zone LB targets (seconds). - **Region fails** → DNS/GSLB or anycast steering (minutes — Module 5). Patterns: active-active (all serve; capacity headroom N+1) vs active-passive (standby; promotion step; risk of “stale standby”).

3. Internal Working

The three hard sub-problems interviewers drill into: 1. **Detection** — health checks, BGP withdrawal, lease/heartbeat expiry; trade speed vs false positives (split brain!). 2. **Redirection** — connection-level (LB reroutes new conns; existing conns *die* and clients must retry) vs routing-level (anycast reconverges) vs DNS (TTL decay). 3. **State** — stateless tiers fail over trivially; databases need replication + promotion (sync vs async = RPO>0 question) and fencing to prevent split brain (only one writer!).

4. Packet Flow Explanation

Zone failover timeline (realistic, tell it like this):

```
t=0      AZ-a networking dies. In-flight requests to AZ-a: hang/reset.
t=0-5s   clients/L7 retries mask some failures (retry budget!)
t=5-30s  LB health checks eject all AZ-a targets; new conns -> AZ-b/c
t=30s+   AZ-b/c absorb +50% load each => THIS is why you run <=66%
         utilization per zone (capacity is a failover feature)
DB tier:  async replica in AZ-b promoted (RPO: last N ms of writes lost),
         fencing token invalidates old primary, clients re-resolve/reconnect.
```

5. ASCII Diagram

```
blast radius ladder & mechanism & speed:
process    supervisor restart      ms-s
backend    LB health check         s      <- capacity headroom!
LB node     ECMP/anycast/VRRP       sub-s..s
zone        multi-AZ LB targets     s-30s
region      DNS/GSLB or anycast     30s-min <- + data replication story
Split brain guard: leases + fencing tokens; only one writer, ever.
```

6. Real Production Example

AWS multi-AZ RDS: synchronous standby, DNS-name flip on failover (~60–120s; clients must re-resolve — ties to Module 5's pooled-connection trap). Route53 + health checks for region failover is the textbook GSLB. Google/Cloudflare regional failover is largely anycast withdrawal — traffic re-routes at BGP speed.

7. Advantages

Automated failover converts hardware/zone mortality into brief latency blips; layered mechanisms keep blast radii independent.

8. Trade-offs

- Capacity cost: surviving N-1 zones at peak means permanent headroom (~33%+ over-provisioning for 3 AZs).
- False-positive failovers cause outages themselves (flappy detection, split brain).
- Async replication = data-loss window (RPO); sync = latency + availability coupling. There is no free lunch — say the CAP-flavored trade explicitly.
- Failover code is the least-tested code in most systems → drills (chaos engineering) are part of the design.

9. Common Mistakes

- Failover plans that assume *clean* failure (crash) — real failures are gray: slow, partial, flapping. Design for “degraded” not just “dead.”
- Forgetting in-flight connections die: failover ≠ transparent; clients need retry logic with idempotency.
- Standby that was never exercised (config drift, cold caches, expired creds) — “the standby always fails the first real failover.”
- No fencing → two primaries → data corruption (worse than downtime).

10. Performance Impact

Quantify per tier: backend failover $\approx$ detection(5–30s) $\times$ error-rate share; regional $\approx$ minutes + RPO. Retry budgets mask short failovers entirely — but unbounded retries turn failover into a retry storm (Module 10). Post-failover latency is elevated (cold caches, TCP slow start on new conns) — mention the *recovery tail*.

11. Common Interview Questions

1. Walk failure of: one pod, one LB node, one AZ, one region — mechanism and user impact for each.
2. Active-active vs active-passive for a database — RPO/RTO trade-offs.
3. How do you prevent split brain during failover? (Quorum, leases, fencing tokens.)

12. Follow-up Questions

- “How do existing TCP connections experience an anycast failover?” $\rightarrow$ mid-flow route change lands packets at a site without the connection state $\rightarrow$ RST/timeout; clients reconnect; QUIC connection IDs mitigate.
- “How do you *test* failover?” $\rightarrow$ game days, chaos (kill zones in staging + prod drills), automated standby validation.

13. Debugging Scenarios

- Failover “worked” but users saw 10 min of errors $\rightarrow$ pooled connections to dead endpoints never re-resolved DNS; fix client re-resolve + server-side RST on old primary.
- After AZ failover, healthy zones degrade $\rightarrow$ insufficient headroom; load shedding kicks in (or should) — capacity, not code.

14. Best Practices

- Health-check-driven LB failover for speed; DNS/GSLB for coarse geography; both layered.
- Run every tier at failover-capable utilization; make retry budgets + idempotency keys standard; drill failovers quarterly; fencing for anything with a single-writer invariant.

15. Practice Questions

1. Design failover for a payment API: 3 regions, Postgres primary in one. Give RTO/RPO per failure class and where you accept data loss vs downtime. (Expected: sync replication in-region, async cross-region; region loss $\rightarrow$ promote with bounded RPO or hold writes: business decision — say it's a business decision.)
2. Your LB tier is 4 boxes behind ECMP. One silently corrupts 1% of packets (gray failure). Why doesn't failover trigger, and what detection would catch it? (Health checks pass; need passive/end-to-end canary probes + per-path error attribution.)

MODULE 6 — One-Page Cheat Sheet

L4 vs L7	L4: per-CONNECTION, 4-tuple hash, no payload, line-rate, DSR possible, protocol-agnostic. gRPC imbalance trap! L7: terminate+parse, per-REQUEST route/retry/pool/observe, CPU cost, XFF/PROXY for client IP. Layer them: L4 -> L7.
REV PROXY	ingress intermediary: TLS, buffering (slow-client isolation), pooling, cache. 3 timeout families; hop-by-hop headers; 502=upstream fail, 504=upstream timeout, 60s stream trap
API GATEWAY	API policy: authn(JWT local verify), per-key rate limits, transforms, BFF aggregation. Keep THIN. North-south only; mesh handles east-west.
STICKY	cookie(L7)/IP-hash(L4, NAT skew)/consistent-hash(legit shards) node death=state loss; prefer stateless + Redis/JWT
HEALTH	liveness(self only!) readiness(deps w/ caps) startup detection = interval x threshold; passive outlier ejection; eject cap + panic threshold prevent self-inflicted outage; slow-start on recovery; shared-dep check = mass-eject bomb
FAILOVER	ladder: LB(s) -> AZ(s-30s) -> region(DNS, min) ; headroom! gray failures, fencing vs split-brain, drill the standby, in-flight conns die -> retries + idempotency required
NUMBERS	L4 ~10-100x L7 throughput L7 +~1ms detection 15-45s typ 3-AZ headroom ~33% Envoy eject cap 10% / panic 50%

MODULE 6 — Top Interview Questions

1. L4 vs L7: what each sees, costs, and when you're *forced* into each.
2. gRPC behind an NLB is imbalanced — mechanism + three fixes.
3. Health-check design such that a DB blip can't eject your whole fleet.
4. Walk a zone failure end-to-end with a realistic timeline and capacity math.
5. Sticky sessions: how, what breaks, the stateless alternative, and the one legitimate modern use (consistent-hash sharding / WebSockets).
6. LB vs reverse proxy vs API gateway vs service mesh — one diagram.
7. How does the client IP survive L4 vs L7 (DSR / PROXY protocol / XFF trust chain)?
8. How do LBs themselves scale and fail over (ECMP, anycast, consistent hashing, VRRP)?

MODULE 6 — Common Mistakes

- Cargo-cult “LB box” with no layer, timeout, or failure story.
- Deep health checks on shared deps; detection math never computed.
- Stickiness as a fix for stateful apps; IP-hash behind CDNs.
- Retries without budgets/idempotency (failover → retry storm).
- Ignoring the LB → backend hop (pools, keep-alive, timeout alignment).
- Standby never drilled; no fencing; assuming failover is invisible to clients.

MODULE 6 — Mock Interview (15 min)

Q1. “Design the ingress path for a 100k-RPS API across 3 AZs.” *Strong answer:* anycast/NLB (L4, ECMP-scaled) → Envoy/ALB tier (L7: TLS, routing, retries with budget, outlier detection) → services; per-AZ target groups with cross-zone off (or on — argue cost vs skew); readiness-gated deploys; capacity 66%/AZ; client IP via PROXY → XFF chain; numbers for detection times and headroom.

Q2. “During a dependency brownout, your fleet started mass-restarting. Postmortem cause?” *Strong answer:* dependency check wired into *liveness* probes → orchestrator restarted healthy pods (restart storm), cold caches worsened brownout → death spiral; fixes: *liveness*=self-only, readiness with eject caps/panic threshold, backoff on restarts, load shedding at the edge.

Q3. “Your canary backend gets 0 traffic behind the LB despite weight 10%. Why?” *Strong answer:* long-lived h2/gRPC connections pinned to old backends (per-connection balancing at L4, or L7 with conn reuse and no rebalance) → set `MAX_CONNECTION_AGE` / connection churn, or per-request L7 weighting; verify with per-backend active-conn and new-conn metrics.